



ค่ายโอลิมปิกวิชาการ
ศูนย์มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

Problem Solving

สถิตย์ ประสมพันธ์

Sathit.p@sci.kmutnb.ac.th



Problem Solving Paradigms

Brute Force

- Iterative Complete Search

- Recursive Complete Search

Divide and Conquer

- Binary Search

- การแปลง Optimization Problem เป็น Decision Problem

Greedy

Dynamic Programming

- Top-down Dynamic Programming (Memorization)

- Bottom-up Dynamic Programming

- Steps to solve Dynamic Programming problems

- Multi-dimensional DP

Brute Force

- Brute Force เป็นวิธีการแก้ปัญหาที่ง่ายที่สุด เพราะไม่จำเป็นต้องคิดวิธีที่ดีอะไรมา เพียงแค่ทดลองทุกคำตอบ/ทุกกรณีที่เป็นไปได้ แล้วตอบคำตอบที่ดีที่สุด
 - หรือตอบจำนวนวิธีที่เป็นไปได้ หากโจทย์ถามหาจำนวนวิธีในการทำอะไรบางอย่าง
- โจทย์ส่วนใหญ่จะให้คะแนนวิธีนี้น้อย แต่ควรศึกษาและฝึก implement Brute Force เพราะหากหาวิธีที่ดีไม่ได้ การทำ Brute Force ก็อาจจะช่วยได้บางส่วน

Iterative Complete Search

- โจทย์บางข้อสามารถ Brute Force ได้โดยการทดลองรูปแบบคำตอบทั้งหมดที่เป็นไปได้ โดยอาจจะลูปหลายชั้น เช่น
- กำหนดลำดับตัวเลขมาให้ n ตัว ให้เลือกตัวเลขสองตัวที่มีค่าต่างกันน้อยที่สุดที่เป็นไปได้
 - วิธีหนึ่งที่ทำได้ คือลูปสองชั้นเพื่อทดลองเลือกตัวเลขสองตัวทุกแบบที่เป็นไปได้ หาวิธีการเลือกที่ได้ค่าแตกต่างกันน้อยที่สุดแล้วตอบ
- กำหนดจำนวนเต็ม A และ B ให้ จงหาจำนวนเต็มบวก x และ y ทุกคู่ที่ตรงตามเงื่อนไข $x+y = A$ และ $x*y = B$
 - ใช้การทดลองลูปสองชั้นเพื่อทดลองค่า x, y ทั้งหมดที่เป็นไปได้ (ไม่เกิน A หรือ B) แล้วตรวจสอบเงื่อนไขทั้งสองเงื่อนไข ถ้าตรงเงื่อนไขก็ตอบคู่ นั้น วิธีนี้จะใช้เวลา $O(\min(A, B)^2)$
 - สังเกตว่าความจริงแล้วเราไม่จำเป็นต้องลูปค่า y ก็ได้ เพราะหากเลือกค่า x แล้ว เราสามารถสรุปได้ทันทีว่า $y = A-x$ ตามสมการแรก สิ่งที่ต้องทำมีเพียงแค่การตรวจสอบเงื่อนไข $x*y = B$ ว่าเป็นจริงหรือไม่ วิธีนี้จะใช้เวลา $O(\min(A, B))$

Recursive Complete Search

- โจทย์บางข้อจำเป็นต้องทดลองวิธีเป็นจำนวนมาก และไม่สามารถใช้ลูปหลายชั้นในการแก้ได้ ดังนั้นจึงต้องเขียน recursive function เพื่อทดลองเลือกทุกวิธีที่เป็นไปได้แทน

Recursive Complete Search

- ต้องการทำอาหาร มีส่วนผสมให้ทั้งหมด n ชิ้น ($n \leq 10$) โดยส่วนผสมแต่ละชิ้นจะมีค่าความเปรี้ยวและค่าความขม ให้เลือกส่วนผสมมาทำอาหาร (เลือกกี่ชิ้นก็ได้ แต่ต้องอย่างน้อย 1 ชิ้น) เมื่อทำอาหาร จะได้อาหารที่มีค่าความเปรี้ยวเท่ากับผลคูณของค่าความเปรี้ยวของส่วนผสมที่เลือกมา ส่วนค่าความขมจะเท่ากับผลบวกของค่าความขมของส่วนผสมที่เลือกมา
- ให้หาวิธีเลือกโดยให้ความต่างระหว่างค่าความขมกับค่าความเปรี้ยวน้อยที่สุดเท่าที่เป็นไปได้

Recursive Complete Search

```
#include <bits/stdc++.h>
using namespace std;
const int N = 20;
const int INF = 1e9;
int n;
int sour[N], bitter[N];
bool picked[N];
int ans = INF;

int main()
{
    scanf("%d", &n);
    for (int i = 1; i <= n; ++i)
        scanf("%d%d", &sour[i], &bitter[i]);
    pick(1);
    printf("%d\n", ans);

    return 0;
}

void pick(int i)
{
    if (i <= n) {
        picked[i] = true;
        pick(i+1);
        picked[i] = false;
        pick(i+1);
    } else { // i == n+1
        int total_sour = 1, total_bitter = 0, count_used = 0;
        for (int j = 1; j <= n; ++j) {
            if (picked[j]) {
                ++count_used;
                total_sour *= sour[j];
                total_bitter += bitter[j];
            }
        }
        if (count_used != 0)
            ans = min(ans, abs(total_sour - total_bitter));
    }
}
```

Divide and Conquer

- แนวคิดหลัก ๆ ของ Divide and Conquer คือ แบ่งปัญหาออกเป็นหลายส่วนที่เท่า ๆ กัน ใช้ อัลกอริทึมเดิมแก้ปัญหาย่อย ๆ ซ้ำแบบ recursive แล้วหาวิธีรวมคำตอบเข้าด้วยกัน เป็น คำตอบของปัญหาใหญ่ (โดยส่วนใหญ่ จะแบ่งออกเป็น 2 ส่วนเท่า ๆ กัน) เช่น
 - Merge Sort ที่ใช้การ sort array ครั้งแรกกับครึ่งหลังแยกกัน แล้วค่อยนำ array ทั้งสอง มารวมกัน เนื่องจากว่ามีการแบ่งครึ่งลงไปทั้งหมด $O(\log n)$ ชั้น แต่ละชั้นจะต้องจัดการกับ สมาชิกของ array เพื่อทำการรวม array - รวมทั้งหมด $O(n)$ ตัว ทำให้เวลาการทำงานของ merge sort เป็น $O(n \log n)$
 - Binary Search บนปัญหา Optimization Problem ที่ถูกแปลงเป็น Decision Problem ซึ่งพบได้บ่อยมาก ควรศึกษาไว้

Binary Search

- กำหนดลำดับจำนวนให้ทั้งหมด n ตัว มีคำถามให้ q คำถาม แต่ละคำถาม ให้หาว่ามีเลข x อยู่ในลำดับหรือไม่
 - เราสามารถ sort array ดังกล่าวเรียงจากน้อยไปมากก่อน แล้วสำหรับแต่ละคำถาม เราจะมีวิธีดังนี้
 - ดูตำแหน่งกลางของ array ก่อน หากตัวเลขตรงกับตัวที่ต้องการหา ก็จบการทำงาน ตอบได้ว่ามีตัวเลขดังกล่าว
 - หากไม่ใช่ ให้ดูว่าตัวเลขที่เราต้องการมีค่าน้อยกว่าหรือมากกว่าตัวเลขตำแหน่งตรงกลาง
 - หากตัวเลขที่เราต้องการมีค่าน้อยกว่า แสดงว่าอาจจะอยู่ทางซ้าย เราจะมาค้นหาในช่วงครึ่งซ้ายแทน แต่หากมากกว่า ให้ค้นหาในช่วงครึ่งขวา
 - การค้นหาแต่ละช่วง จะใช้ดูค่าตรงกลาง แล้วเลือกครึ่งที่สมควรค้นหาเรื่อย ๆ เช่นเดียวกัน
 - แต่ละครั้ง array ของเราจะถูกตัดออกไปทีละครึ่ง ดังนั้น รวมแล้วจะใช้เวลาไม่เกิน $O(\log n)$ ในการตอบคำถามแต่ละครั้ง



การแปลง Optimization Problem เป็น Decision Problem

- ปัญหาที่ต้องการให้หาค่าน้อยสุด/มากที่สุดบางอย่าง สามารถแปลงมาเป็นปัญหาการตัดสินใจ (ที่ต้องตอบ Yes/No) เพื่อที่จะทำให้หาคำตอบโดยการ binary search ได้ ทำให้แก้ปัญหาดังนี้ง่ายขึ้น ยกตัวอย่างดังนี้

การแปลง Optimization Problem เป็น Decision Problem

- มีลำดับจำนวนเต็มบวกอยู่ n ตัว ต้องการแบ่งเป็นช่วง ๆ โดยจะมีค่า x กำหนดไว้ แต่ละช่วงจะต้องมีผลรวมของสมาชิกไม่เกิน x ถามว่า หากเราต้องการแบ่งออกเป็นช่วงไม่เกิน k ช่วง เราสามารถกำหนดค่า x ให้น้อยสุดได้เท่าไร
- ตัวอย่าง 3, 5, 1, 2, 4 และ $k = 2$
 - หากเรากำหนดให้ $x = 7$ จะพบว่า การแบ่งโดยให้แต่ละช่วงมีผลรวมไม่เกิน $x = 7$ จะต้องใช้อย่างน้อย 3 ช่วง เช่น [3], [5], [1, 2, 4] - ค่า x นี้ใช้ไม่ได้ เพราะเราต้องการแบ่งเป็นช่วงไม่เกิน $k = 2$ ช่วง
 - หากเรากำหนดให้ $x = 8$ พบว่าเราสามารถแบ่งเป็น [3, 5], [1, 2, 4] ซึ่งใช้เพียง $k = 2$ ช่วงเท่านั้น ไม่ขัดกับเงื่อนไขโจทย์
- หากเราเปลี่ยนปัญหาจาก "หาค่า x ที่น้อยสุดที่เป็นไปได้" เป็น "ตรวจสอบว่า ถ้าแบ่งให้แต่ละช่วงมีผลรวมไม่เกิน x จะแบ่งให้มีไม่เกิน k ช่วงได้หรือไม่" เราจะตอบปัญหาแบบหลังได้ง่ายขึ้น

การแปลง Optimization Problem เป็น Decision Problem

- การตรวจสอบว่าทำได้หรือไม่ สามารถทำได้แบบ greedy
 - เริ่มต้นมา ให้ช่วงแรกประกอบด้วยจำนวนตัวแรกเพียงตัวเดียว
 - พยายามเพิ่มตัวถัดไปเรื่อย ๆ トラバタイที่ผลรวมยังไม่เกิน x
 - ถ้าเกินแล้ว ให้เริ่มต้นช่วงใหม่
 - ทำไปเรื่อย ๆ จนครบทุกตัว แล้วตรวจสอบว่าจำนวนช่วงเกิน k หรือไม่
- เมื่อเราสามารถตรวจสอบคำตอบได้อย่างง่ายดาย การที่จะแก้ปัญหาดังเดิมได้นั้น ก็เพียงแค่ binary search บนค่า x โดยมีเงื่อนไขดังนี้
 - เริ่มต้นมา พิจารณาค่า x ที่เป็นไปได้ตั้งแต่ 1 ถึง INF (เมื่อ INF คือ bound ผลรวมที่มากที่สุดที่เป็นไปได้ของ array ทั้ง array)
 - ทดลองค่า x ตรงกลางช่วง หากตรวจสอบแล้วสามารถทำให้เงื่อนไขเป็นจริงได้ สังเกตว่าถ้าเราเพิ่มค่า x ยิ่งเงื่อนไขก็ยิ่งเป็นจริงอยู่ แต่เราต้องการค่า x น้อยสุด ดังนั้น เราจะพิจารณาเฉพาะช่วงคำตอบ x ตั้งแต่ 1 ถึงค่าดังกล่าว (เพื่อให้ x น้อยกว่านี้ได้อีก)
 - หากทดลองแล้วพบว่าเงื่อนไขเป็นเท็จ นั้นแปลว่าเราจำเป็นต้องเพิ่มค่า x เพราะฉะนั้น เราจะพิจารณาค่า x ที่เป็นไปได้ ตั้งแต่ ค่าดังกล่าวบวก 1 ถึง INF
 - เราจะแบ่งครึ่งช่วงไปเรื่อย ๆ จนกว่าจะเหลือเพียงค่าเดียว ได้เป็นคำตอบของคำถามดังกล่าว

การแปลง Optimization Problem เป็น Decision Problem

```
#include <bits/stdc++.h> //
<- include everything
using namespace std;
```

```
const int N = 100010;
const long long INF = 1e18;
```

```
int n, k;
long long val[N];
```

```
int main()
{
    scanf("%d%d", &k, &n);
    for (int i = 1; i <= n; ++i)
        scanf("%lld", &val[i]);

    long long lo = 1;
    long long hi = INF;
    while (lo < hi) {
        long long mid = (lo+hi)/2;
        if (check(mid))
            hi = mid;
        else
            lo = mid+1;
    }
    printf("%lld\n", lo);

    return 0;
}
```

```
bool check(long long limit)
{
    int cnt = 1;
    long long sum = 0;
    for (int i = 1; i <= n; ++i) {
        if (val[i] > limit)
            return false;
        // เริ่มต้นช่วงใหม่ ถ้าเพิ่มตัวใหม่เข้าไปแล้วทำให้ผลรวมเกิน
        if (sum + val[i] > limit) {
            ++cnt;
            sum = 0;
        }
        sum += val[i];
    }
    return cnt <= k;
}
```

Greedy

- Greedy Algorithm คือขั้นตอนวิธีที่ใช้วิธีการเลือกตัวเลือกที่ดีที่สุด เท่าที่เห็น ณ ตอนนั้นโดยไม่คำนึงถึงว่าการเลือกตัวเลือกดังกล่าวจะทำให้คำตอบนั้นดีที่สุดหรือไม่
- โดยปกติจะใช้ Greedy Algorithm ในกรณีที่พิสูจน์ได้ว่า ถึงเลือกตัวเลือกไปแบบไม่คิดอะไรมาก ณ ตอนนั้น ทำเช่นนี้ไปเรื่อย ๆ ย่อมได้คำตอบที่ดีที่สุดเสมอ
- ตัวอย่าง Greedy Algorithm อย่าง Dijkstra's Algorithm, Kruskal's Algorithm, Prim's Algorithm

Dynamic Programming

- **Dynamic Programming (DP)** คือการแก้ปัญหามีลักษณะคล้าย ๆ การเขียนฟังก์ชัน brute force ที่มีการ return คำตอบ เพียงแต่อาศัยคุณสมบัติพิเศษของปัญหา เพื่อให้การคิดคำนวณทำได้ไวขึ้น หลัก ๆ แล้วมีสองอย่าง ได้แก่
 - **Optimal Substructure** หากสร้างคำตอบของปัญหาใหญ่ได้ คำตอบของปัญหาย่อยย่อมดีที่สุดด้วย
 - **Overlapping Subproblems** ปัญหาใหญ่สามารถแบ่งเป็นปัญหาย่อยต่าง ๆ ได้ แล้วปัญหาย่อยต่าง ๆ หากแบ่งออกไปอีก อาจจะแบ่งได้ปัญหาย่อยที่ซ้ำกัน
- ถ้าเจอปัญหาซ้ำกัน เราไม่จำเป็นต้องคิดคำตอบซ้ำ ทำให้สามารถคำนวณคำตอบของปัญหาทั้งหมดได้เร็วขึ้น
- เทคนิค Dynamic Programming นี้สามารถนำไปใช้ได้กับ Optimization Problem, Counting Problem

Fibonacci Numbers

การหาตัวเลข Fibonacci ตัวที่ n โดยตัวเลข Fibonacci มีนิยามดังนี้

$$F[0] = F[1] = 1$$

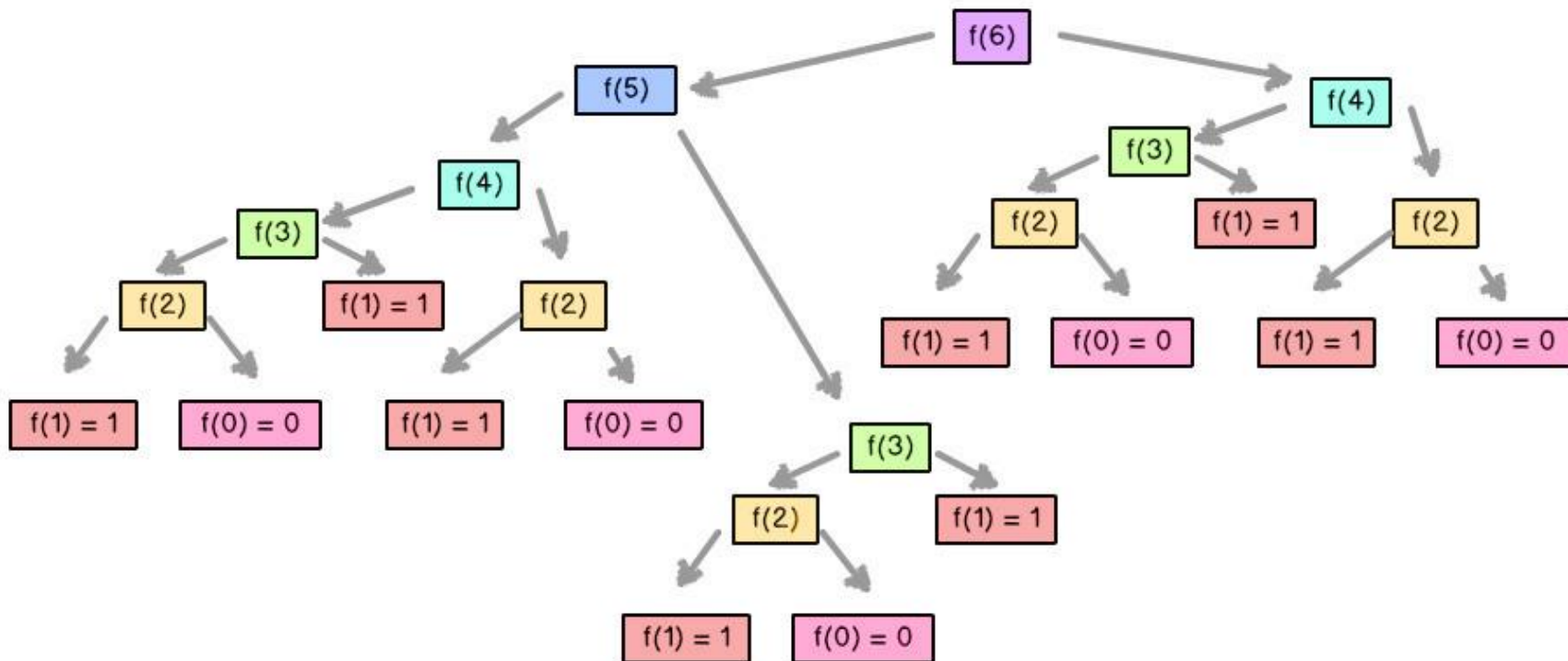
$$F[n] = F[n-1] + F[n-2]$$

หากเขียนเป็น recursive function จะได้โค้ดดังนี้

```
int fibonacci(int n) {  
    if (n == 0 || n == 1)  
        return 1;  
    return fibonacci(n-1) + fibonacci(n-2);  
}
```

Fibonacci Numbers

- พิจารณารูปด้านล่าง สังเกตได้ว่าหากคำนวณโดยใช้ recursive function จะมีค่า $F[i]$ บางค่าที่ถูกคำนวณซ้ำ รวมแล้วพบว่าจะใช้เวลาในการทำงานเป็น $O(2^n)$



Top-down Dynamic Programming (Memorization)

- หนึ่งในวิธีการหลีกเลี่ยงการคิดคำนวณซ้ำคือการทำ memorization
- การเขียนฟังก์ชัน recursive ที่ใส่ break condition เพิ่มเติม โดยตรวจสอบว่าเคยเข้าถึง state นั้นหรือยัง หากพบว่าเคยเข้าถึง state นั้นแล้ว ก็จะ return คำตอบที่เคยคำนวณไว้ทันทีเลย
- สังเกตว่าเราจะเรียกฟังก์ชันเพื่อแก้ปัญหาลูกก่อน แล้วฟังก์ชันนั้นจะเรียกตนเองแบบ recursive เพื่อแก้ปัญหาย่อยเรื่อย ๆ ดังนั้นวิธีการ implement DP แบบดังกล่าวจึงเรียกว่าเป็นแบบ Top-down
- ตามตัวอย่างการคำนวณค่า Fibonacci โค้ดจะมีลักษณะดังนี้

Fibonacci Numbers

```
int F[N];
bool visited[N];

int fibonacci(int n) {
    if (n == 0 || n == 1)
        return 1;
    if (visited[n]) // return if already computed
        return F[n];
    visited[n] = true; // set as "already computed"
    F[n] = fibonacci(n-1) + fibonacci(n-2); // remember answer
    return F[n];
}
```

สังเกตว่าอัลกอริทึมดังกล่าวจะใช้เวลา $O(n)$ เพราะการใส่ break condition ทำให้มี state เพียง $O(n)$ state เท่านั้น สำหรับค่า n แต่ละค่า หากเราทราบคำตอบของค่า n ที่น้อยกว่าอยู่แล้ว จะสามารถคำนวณคำตอบได้ใน $O(1)$ (นั่นคือนำคำตอบมาบวกกันเฉย ๆ) อย่างไรก็ตาม การ implement แบบ top-down โค้ดอาจจะทำงานได้ช้า เนื่องจาก constant factor ในการจัดการการเรียกฟังก์ชันของระบบสูง

Bottom-up Dynamic Programming

- อีกวิธีที่คนมักใช้กันก็คือ Bottom-up ซึ่งเป็นการหาคำตอบของ state ที่อยู่ด้านล่างก่อน (ในกราฟของ state) โดยจะเริ่มจาก base case ที่รู้คำตอบอยู่แล้ว แล้วจะค่อย ๆ สร้างของ state ที่ใหญ่ขึ้นไปเรื่อย ๆ
- โค้ดของ Fibonacci ที่ implement แบบ Bottom-up จะเป็นดังนี้

```
int fibonacci(int n) {  
    int F[n+1];  
    F[0] = F[1] = 1;  
    for (int i = 2; i <= n; i++)  
        F[i] = F[i-1] + F[i-2];  
    return F[n];  
}
```

Steps to solve Dynamic Programming problems

- ขั้นตอนในการแก้โจทย์ DP โดยจะใช้ปัญหา Coin Change เป็นตัวอย่าง
 - กำหนดจำนวนเต็ม n แทนจำนวนประเภทของเหรียญ และ k แทนมูลค่าเงินที่ต้องการ มีลำดับ C โดยที่ $C[i]$ แทนค่าของเหรียญประเภทที่ i
 - สมมติให้เหรียญแต่ละประเภทมีจำนวนไม่จำกัด จงหาว่าจะต้องใช้เหรียญน้อยสุดกี่เหรียญจึงจะได้เงินรวมเท่ากับ k พอดี
- สิ่งแรกที่เราต้องทำเมื่อเราคิดว่าโจทย์สามารถแก้ได้โดยใช้ Dynamic Programming คือสร้างสมการ recursive กำกับคำตอบ
 - ถ้าเราสามารถสร้างสมการ recursive นี้ได้ เราก็สามารถใช้ Top-down หรือ Bottom-up เพื่อให้สามารถคำนวณได้รวดเร็ว
- การสร้างสมการขึ้นมาเป็นสิ่งที่ต้องอาศัยการฝึกฝนอย่างสูง โดยเฉพาะข้อที่ยาก ๆ และต้องใช้สมการแปลก ๆ อย่งไรก็ตาม มันจะง่ายขึ้นถ้าเราคิดตามสามขั้นตอนนี้



Steps to solve Dynamic Programming problems

- หาว่าเราควรจะกำหนด state อะไร และ state นั้นบ่งบอกถึงอะไร
- หาว่า state นั้นจะสามารถสร้างคำตอบจาก state ใดได้บ้าง
- หาว่าเราจะคำนวณคำตอบของ state นั้นจาก state อื่นอย่างไร

Steps to solve Dynamic Programming problems

- สำหรับโจทย์ Coin Change การกำหนด state สามารถทำได้โดยสังเกตว่าโจทย์กำหนดอะไรให้บ้าง หรือกำหนดให้หาอะไรบ้าง ในที่นี้ พบว่ามี
 - จำนวนประเภทของเหรียญ
 - มูลค่าเงินที่ต้องการ
 - มูลค่าของประเภทเหรียญแต่ละเหรียญ
 - จำนวนเหรียญที่น้อยที่สุดที่ต้องใช้ (สิ่งที่โจทย์ให้หา)
- หากลองกำหนดให้ state ขึ้นอยู่กับมูลค่าของเงินที่ต้องการแทน เราก็สามารถกำหนดให้ $dp[i]$ เท่ากับจำนวนเหรียญที่น้อยที่สุดที่ต้องใช้เพื่อให้ได้มูลค่ารวม i บาท สังเกตว่าการกำหนดแบบนี้จะตรงกับสิ่งที่โจทย์ต้องการให้หาพอดี หากเราต้องการเงิน k บาทก็สามารถตอบจาก $dp[k]$ ได้เลย

Steps to solve Dynamic Programming problems

- ถ้าเราต้องการเงิน x บาท เราอาจจะเลือกใช้เหรียญใดเหรียญหนึ่งก็ได้
 - เช่น หากเลือกใช้เหรียญ $C[i]$ นั้นแปลว่า เรามี 1 เหรียญที่มีมูลค่า $C[i]$ บาท และเงินอีก $x - C[i]$ บาทที่ต้องพิจารณาว่าจะใช้เหรียญใด สังเกตว่าในส่วน $x - C[i]$ นั้นเป็นปัญหาย่อยลักษณะเดียวกัน
 - เราสามารถใช้ฟังก์ชัน recursive เพื่อคำนวณว่ามูลค่า $x - C[i]$ ต้องใช้เหรียญน้อยสุดเท่าใด เนื่องจากว่าเราต้องการทอนเงิน x บาทโดยใช้จำนวนเหรียญน้อยที่สุดที่เป็นไปได้ เราต้องทดลองค่า i ตั้งแต่ 1 ถึง n เพื่อหาจำนวนเหรียญที่น้อยที่สุด
- ดังนั้น สมการเวียนเกิดของ dp ของโจทย์ข้อนี้คือ
 - $dp[0] = 0$ // เงิน 0 บาท ไม่จำเป็นต้องใช้เหรียญ
 - $dp[x] = \min(dp[x - C[i]] + 1, 1 \leq i \leq n \text{ and } C[i] \leq x)$
- // มีเงิน x บาท หากสร้างจากการใช้เหรียญ $C[i]$ ต้องพิจารณาแก้ปัญหากรณีที่มีเงิน $x - C[i]$ บาทอีก แล้วมารวมกับเหรียญ $C[i]$ 1 เหรียญ ดังนั้น จะได้คำตอบรวมเป็น $dp[x - C[i]] + 1$
- โดยที่ x คือราคาของใน state นั้นๆ และคำตอบสุดท้ายคือ $dp[k]$



Steps to solve Dynamic Programming problems

```
int n, C[N], dp[N];
bool visited[N];

int coin_change(int x) {
    if (x == 0)
        return 0;
    if (visited[x])
        return dp[x];
    visited[x] = true;
    dp[x] = INF;
    for (int i = 1; i <= n; i++) {
        if (C[i] <= x)
            dp[x] = min(solve(x - C[i]) + 1, dp[x]);
    }
    return dp[x];
}

// answer: coin_change(k)
```

```
int n, C[N], dp[N];
int coin_change(int k) {
    fill(dp, dp+N, INF);
    dp[0] = 0;
    for (int x = 1; x <= k; x++) {
        for (int i = 1; i <= n; i++) {
            if (C[i] <= x)
                dp[x] = min(dp[x - C[i]] + 1, dp[x]);
        }
    }
    return dp[k];
}
```

แต่ถ้าเราเขียนแบบ Bottom-up ก็จะเป็นแบบนี้

การเขียนในลักษณะของ Top-down ก็จะได้โค้ดดังนี้



Steps to solve Dynamic Programming problems

- สังเกตว่า แต่ละ state เราต้องลูปค่า i เพื่อหาจำนวนเหรียญที่น้อยที่สุด ซึ่งมีค่า i ที่เป็นไปได้ $O(n)$ แบบ เนื่องจากมีทั้งหมด $O(k)$ state รวมแล้วจะใช้เวลา $O(nk)$
- นอกจากนี้ปัญหา Coin Change อาจจะเปลี่ยนคำถาม แทนที่จะให้หาจำนวนเงินน้อยสุด อาจจะถามจำนวนลำดับวิธีการเลือกเหรียญให้มีมูลค่ารวม k บาทก็ได้ เช่น หาก $n = 2$, $C = [1, 2]$ และ $k = 4$ ก็จะได้ 5 รูปแบบดังนี้
- $4 = 1+1+1+1$
- $4 = 1+1+2$
- $4 = 1+2+1$
- $4 = 2+1+1$
- $4 = 2+2$
- ในที่นี้เราสามารถเปลี่ยนสมการเวียนเกิดเป็น
- $dp[0] = 1$ // การทอนเงิน 0 บาทสามารถทำได้ 1 แบบคือไม่ใช้เหรียญเลย
- $dp[x] = \text{sum}(dp[x-C[i]], 1 \leq i \leq n \text{ and } C[i] \leq x)$
- // การทอนเงิน x บาท สามารถทำได้โดยเลือกเหรียญแรกที่จะใช้ก่อน สมมุติว่าเลือกเหรียญ $C[i]$, แล้วพิจารณานับจำนวนวิธีที่จะทอนเงิน $x-C[i]$ บาท

Multi-dimensional DP

- สังเกตว่าในตัวอย่างก่อนหน้านี้ state dp ของเราจะเป็น array 1 มิติ แต่จริง ๆ แล้วหาก state ของเราต้องเก็บข้อมูลมากกว่านั้น เราอาจจะใช้ตาราง 2 มิติขึ้นไปก็ได้
- Maximum Value Monotonic Path
- กำหนดตารางขนาด $n \times m$ ให้ โดยช่อง $A[i][j]$ จะมีจำนวนเต็ม ให้หาเส้นทางเดินจากตำแหน่ง $(1, 1)$ ซึ่งเป็นตำแหน่งมุมบนซ้าย มายังตำแหน่ง (n, m) ซึ่งเป็นตำแหน่งมุมล่างขวา โดยอนุญาตให้เดินขวาหรือลงเท่านั้น (ห้ามเดินขึ้นหรือซ้าย) เมื่อเดินเสร็จแล้วจะได้มูลค่าเท่ากับผลรวมของค่า $A[i][j]$ ที่เดินผ่านทั้งหมด ถามว่ามูลค่าทางเดินที่มากที่สุดที่เป็นไปได้เท่ากับเท่าใด
 - สังเกตว่าโจทย์ข้อนี้ ตำแหน่งที่กำหนดให้มาเป็นตำแหน่ง 2 มิติ ดังนั้น เราอาจจะให้ฟังก์ชัน/state ของเรานิยามเป็นตาราง 2 มิติก็ได้ โดยกำหนดให้ $dp[i][j]$ เมื่อ $1 \leq i \leq n$ และ $1 \leq j \leq m$ มีค่าเท่ากับ ผลรวมที่มากที่สุดที่เป็นไปได้ เมื่อเดินจากช่อง $(1, 1)$ มาถึงช่อง (i, j)
 - สังเกตว่าการที่จะเดินมาถึงช่อง (i, j) ได้ เราจำเป็นต้องเดินมาถึงช่อง $(i-1, j)$ ก่อนแล้วเดินลง หรือเดินมาถึงช่อง $(i, j-1)$ ก่อนแล้วเดินขวา เราจะพิจารณาทั้งสองวิธีที่เป็นไปได้แล้วหาว่าวิธีใดให้ผลรวมมากกว่ากัน จะได้สมการเวียนเกิดดังนี้
 - $dp[1][1] = A[1][1]$ // base case คือกรณีที่จุดเริ่มต้นตรงกับจุดจบ
 - $dp[0][j] = dp[i][0] = -\text{INF}$ // ไม่อนุญาตให้เดินออกขอบ
 - $dp[i][j] = \max(dp[i-1][j], dp[i][j-1]) + A[i][j]$
 - คำตอบสุดท้ายจะอยู่ที่ $dp[n][m]$
 - สังเกตว่ามี state ทั้งหมด nm state และแต่ละ state สามารถคำนวณคำตอบได้ใน constant time ดังนั้น รวมแล้วอัลกอริทึมนี้จะใช้เวลา $O(nm)$

Multi-dimensional DP

```
const int INF = 1e9;
for (int i = 1; i <= n; ++i)
    dp[i][0] = -INF;
for (int j = 1; j <= m; ++j)
    dp[0][j] = -INF;
for (int i = 1; i <= n; ++i) {
    for (int j = 1; j <= m; ++j) {
        if (i == 1 && j == 1)
            dp[i][j] = A[i][j];
        else
            dp[i][j] = max(dp[i-1][j], dp[i][j-1]) + A[i][j];
    }
}
// answer: dp[n][m]
```

ทดลองทำโจทย์